DECK 9 OF 12 · FRC JAVA CRASH COURSE

Advanced controls

Driver feel, drivetrain feedforward, where the loop runs, and smoother motion

WPILib 2026 · Java · Command-based

OVERVIEW

What this deck covers

Shape the stick

A deadband, a curve and a slew-rate limit, in the right order.

Predict the volts

kS, kV and kA on a drivetrain, and what SysId measures.

Place the loop

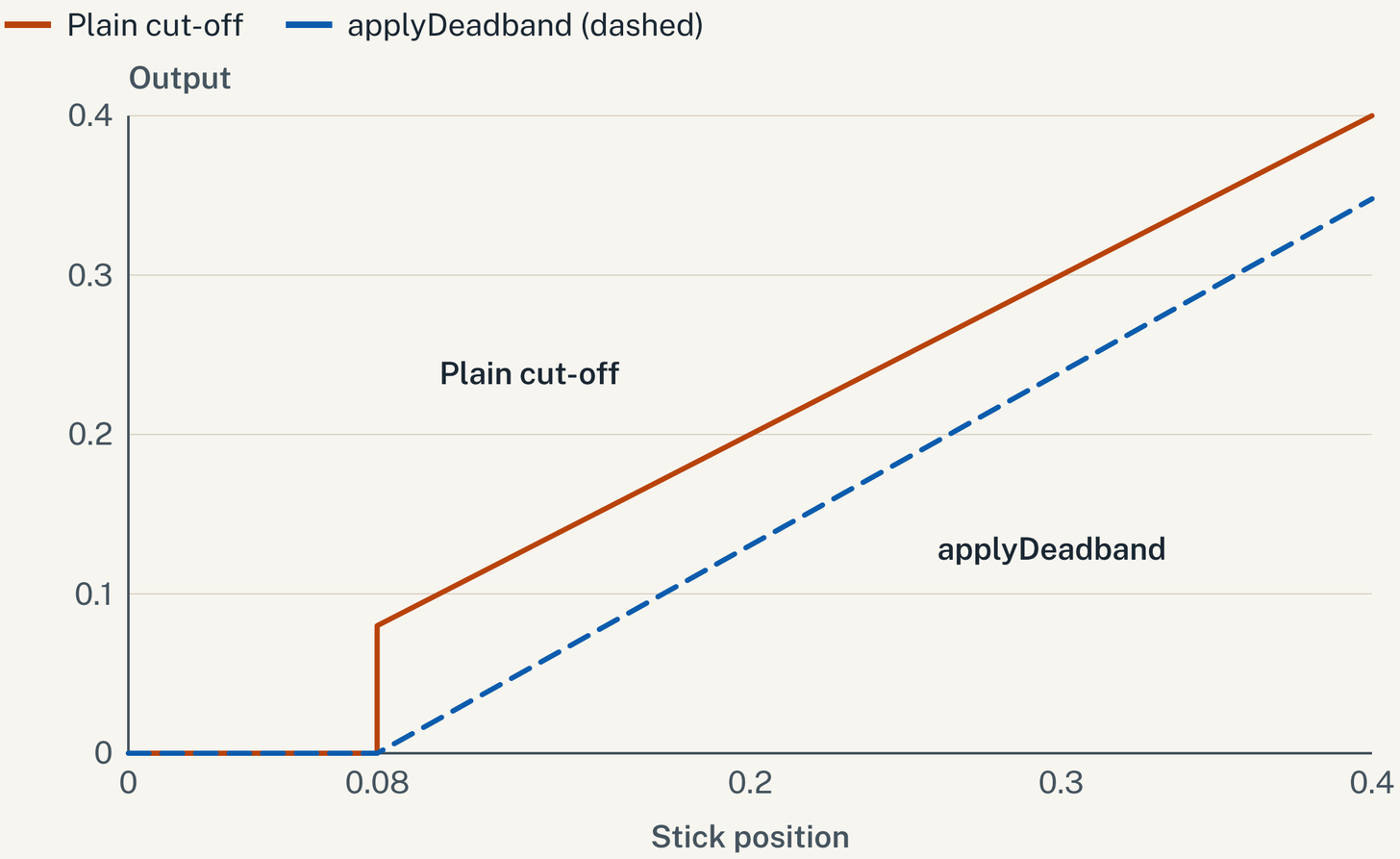
20 ms on the roboRIO or 1 ms on the controller.

Smooth the motion

S-curves, trapezoids and what comes after PID.

CONTROL THEORY

A deadband ignores a stick that rests off zero



1. **What it does**
   Anything under 0.08 becomes 0, so a resting stick can’t creep the robot.

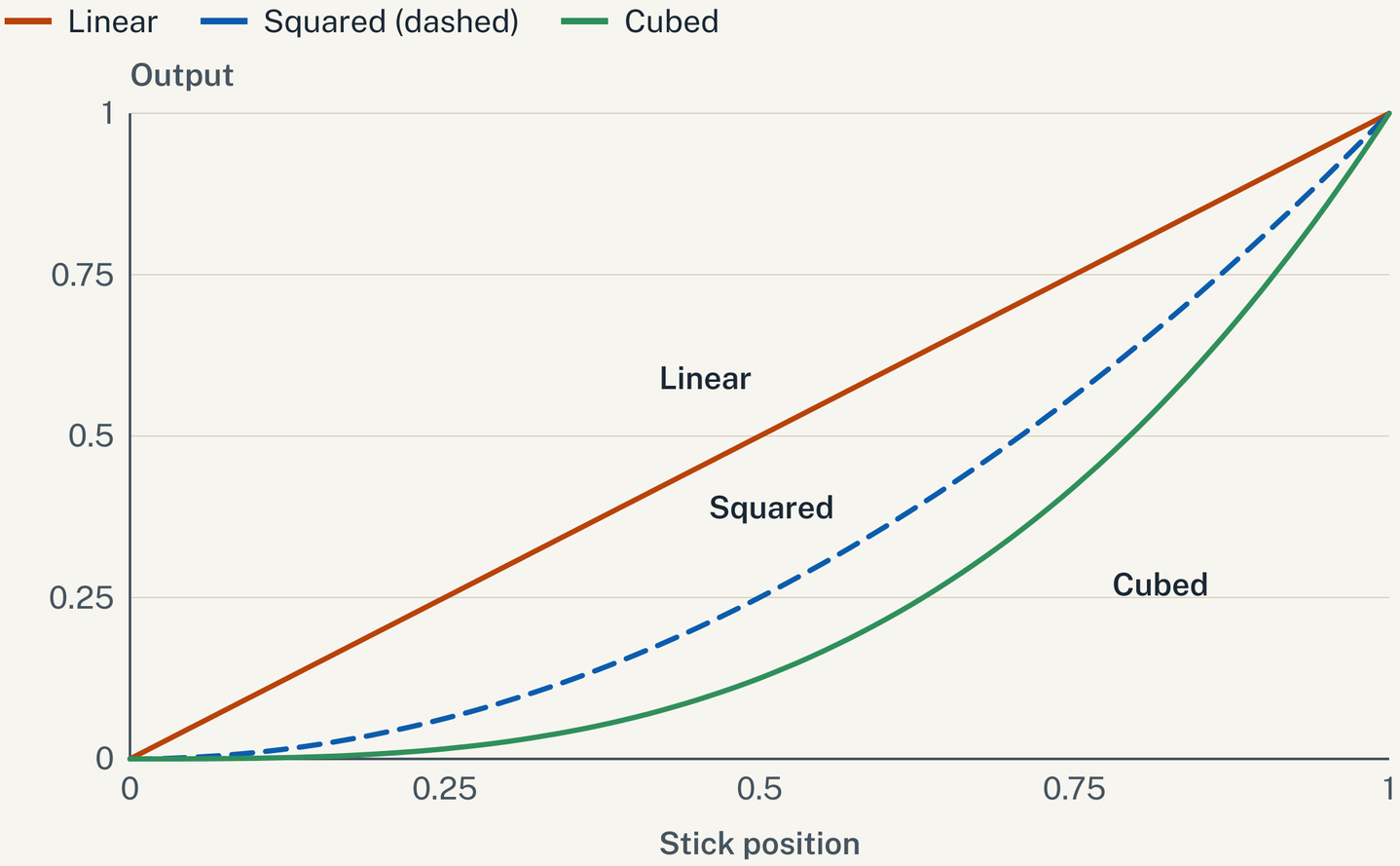
2. **No jump**
   Past 0.08, applyDeadband starts again from 0.

3. **The cut-off**
   A plain if leaps from 0 to 8% of full output.

4. **Full stick**
   Both still reach 1 at the top of the stick.

CONTROL THEORY

Input shaping gives the stick fine control



1. **Half stick**
   Linear gives 50%, squared 25% and cubed 12.5%.

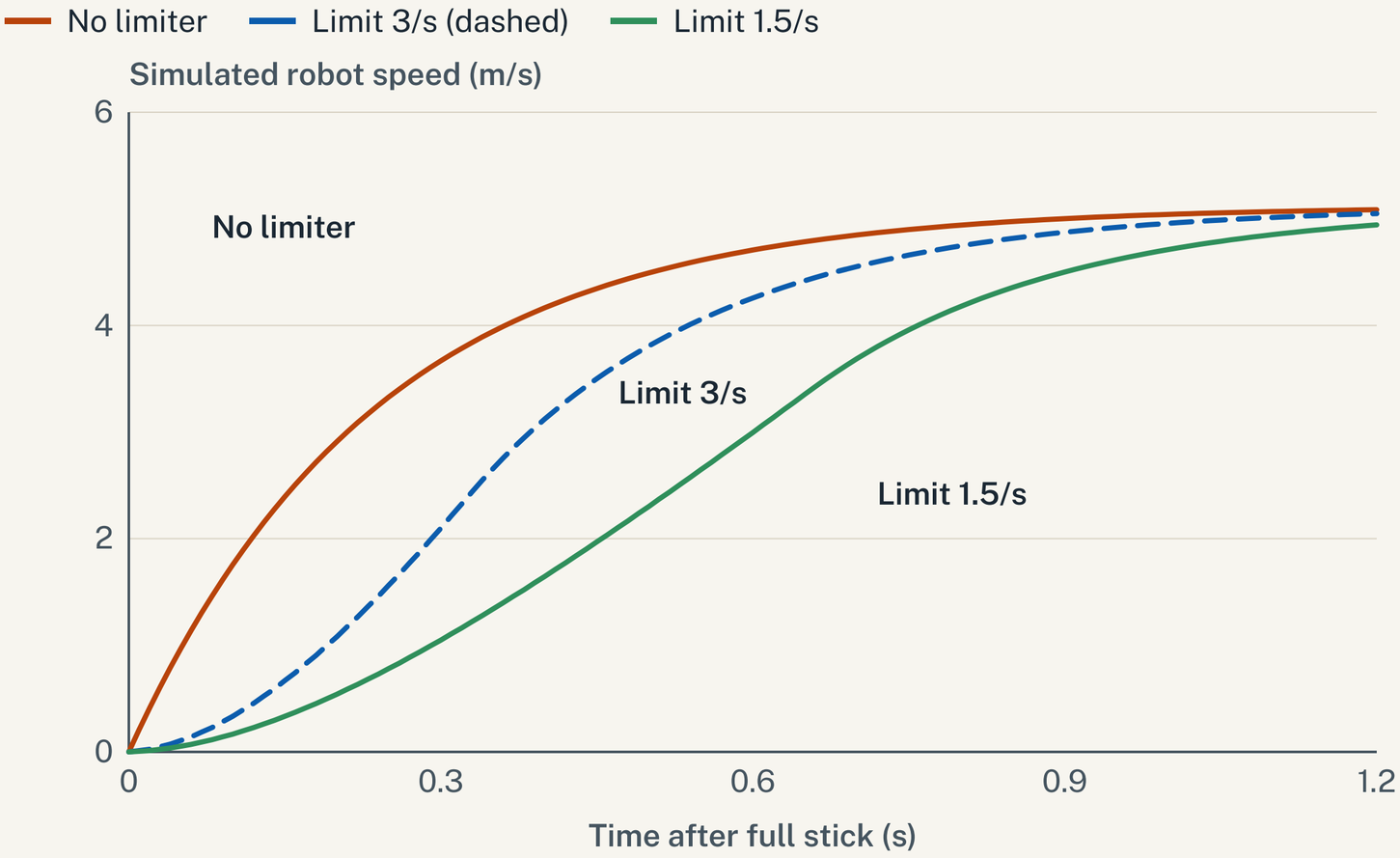
2. **Full stick**
   All three still reach 100%, so you keep your top speed.

3. **Already squared**
   arcadeDrive squares its inputs unless you pass false.

4. **Scale goes first**
   Deck 2’s is squared too: 0.7 becomes 0.49 at full stick.

CONTROL THEORY

A slew-rate limit trades response for smoothness



1 What it does
SlewRateLimiter(3) lets the value change 3 per second: 0 to full in 1/3 s.

2 Peak current
133 A, then 73 A, then 46 A per CIM.

3 Why it feels better
Lower acceleration means less wheel slip and less tipping.

4 The cost
The robot responds later, and that includes stopping.

CONTROL THEORY

The order of the stick steps matters

FeelRobotContainer.java

```
  private double shapedForward() {
    double x = -driver.getLeftY();
1   x = MathUtil.applyDeadband(x, DEADBAND);
2   x = MathUtil.copyDirectionPow(x, STICK_EXPONENT);
3   return forwardLimiter.calculate(x * DRIVE_SCALE);
  }

  private double shapedTurn() {
    double x = -driver.getRightX();
1   x = MathUtil.applyDeadband(x, DEADBAND);
2   x = MathUtil.copyDirectionPow(x, STICK_EXPONENT);
3   return x * TURN_SCALE;
  }
```

1. **Deadband first**
   Throw away drift before anything else sees it.

2. **Then the curve**
   copyDirectionPow keeps the sign, so reverse works.

3. **Scale, then limit**
   Now 0.7 stays 0.7 at full stick, and the limiter sees the final value.

EXERCISE

Tune the drive feel

1. Add the shaping from the last slide, using MathUtil.copyDirectionPow.
2. Stop it being squared twice: call drive.arcadeDrive(x, z, false).
3. Try exponents 1, 2 and 3. Creep along a tape line with each one.
4. Add new SlewRateLimiter(3) on forward, then try 1.5. Watch the stop.
5. Put your best numbers in Constants, then commit.

Done when: the robot creeps 2 m down the tape and stops within 10 cm of the mark.

CONTROL THEORY

Three gains predict the volts a drivetrain needs

Volts = kS + kV × speed + kA × acceleration. Deck 6 did this for an arm.

kS: friction

The volts needed to start moving. We assume 1 V.

kV: speed

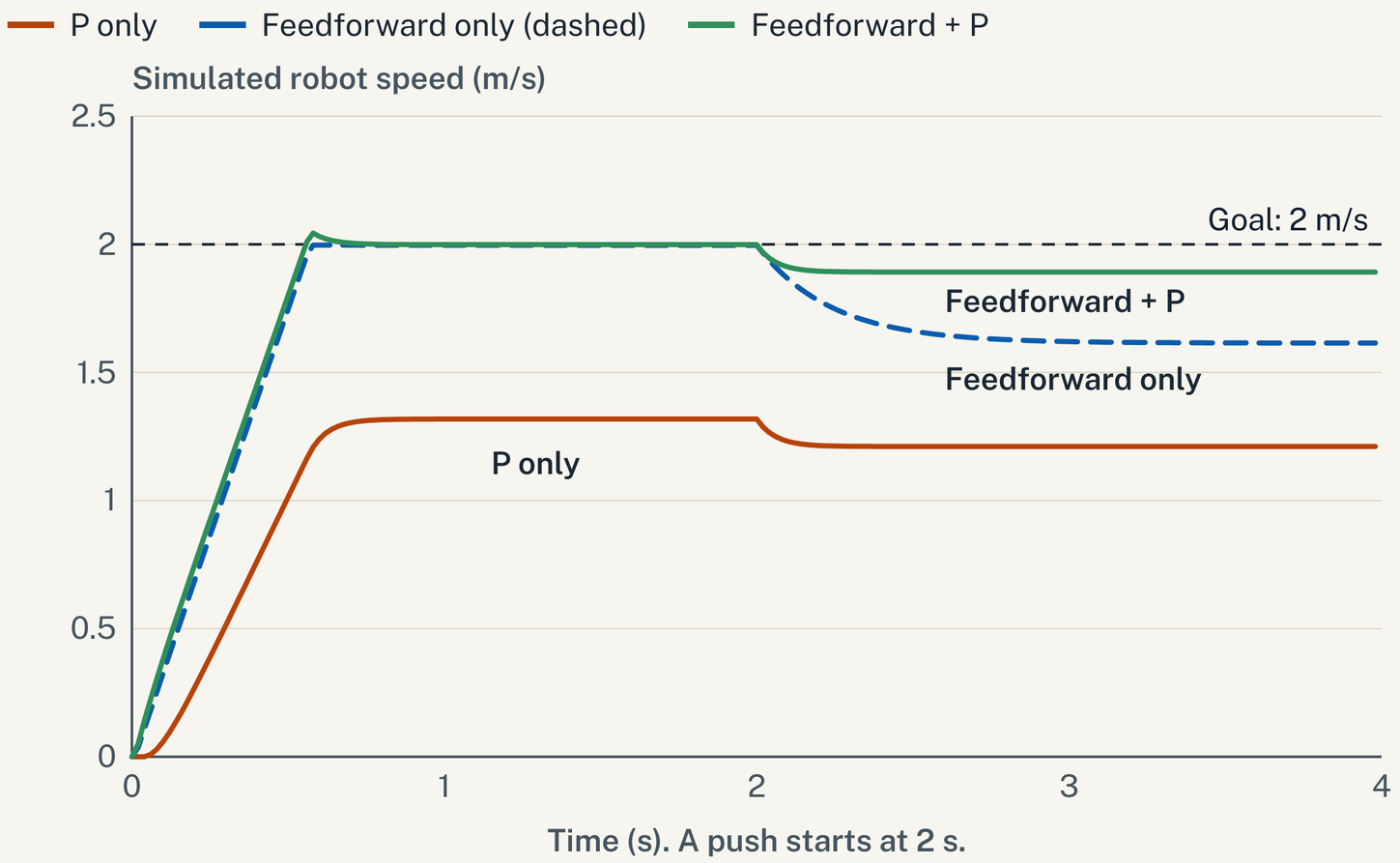
Back-EMF grows with speed, so speed costs volts: 2.34 V per m/s.

kA: acceleration

The extra volts needed while gaining speed: 0.56 V per m/s^2.

CONTROL THEORY

Feedforward does the work and P fixes the miss



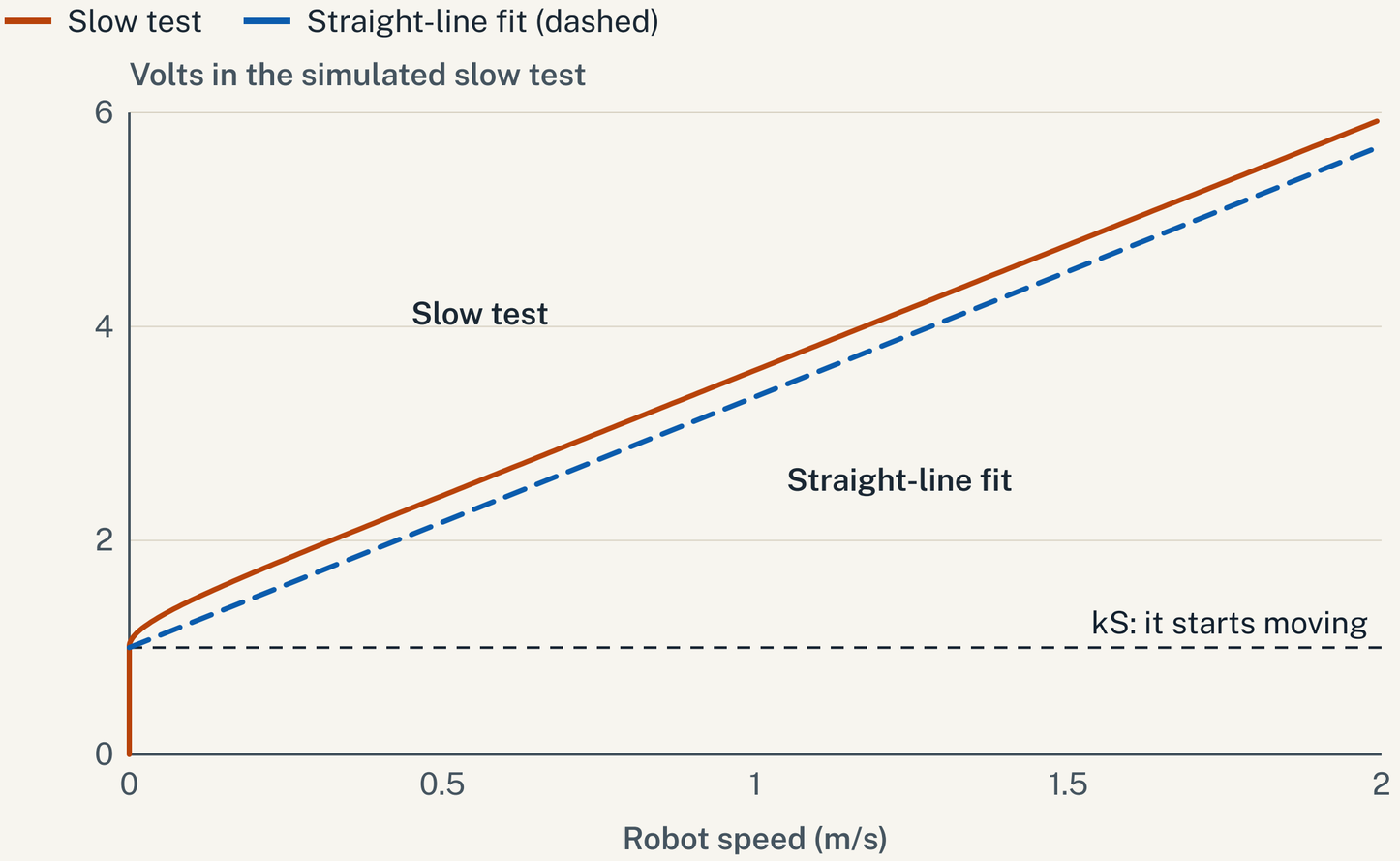
1 P only
Stops at 1.32 m/s. P makes no volts unless it is still wrong.

2 Feedforward only
On the goal, but it can’t see the push: 1.61 m/s.

3 Both
Feedforward carries the speed and P fights the push: 1.89 m/s.

CONTROL THEORY

SysId measures your gains from two simple tests



1 The slow test
Volts creep up 1 V per second, so almost none go to speeding up.

2 Slope: kV
Each m/s took 2.34 V. The model's true value is 2.34.

3 Start: kS
The line starts at 1.00 V. The true value is 1.00.

4 The fast test
A 7 V step shows the extra volts to speed up: kA 0.56.

PROGRAMMING

One speed loop runs each side of the drivetrain

SpeedLoop.java

```
  /** Call once per loop. Returns the volts for this side. */
  public double calculate(double measured, double goal) {
    // The volts we can predict: kS, kV and kA.
1   double ffVolts =
1       feedforward.calculateWithVelocities(lastGoal, goal);
    // The volts we couldn't: P works on what is left over.
2   double pidVolts = pid.calculate(measured, goal);
3   lastGoal = goal;
4   return MathUtil.clamp(
4       ffVolts + pidVolts, -MAX_VOLTS, MAX_VOLTS);
  }
```

1. **Feedforward**
   Two goals in a row give it the speed and the acceleration.

2. **P**
   Works on what is left. KP is 6 volts per m/s of error.

3. **Remember the goal**
   This is why each side gets its own object.

4. **Add, then clamp**
   Volts add together, and a motor only gets 12.

COMMON MISTAKE

One side of the drive runs away

What you see	You enable the speed loop, and one side slams to full speed. The robot spins or curves hard.
Why it happens	The sides are mirror images, so one encoder counts backward. Speeding up looks like slowing down, so P pushes harder.
The fix	Disabled, wheels up: turn each wheel forward by hand. Both speeds must rise. If one falls, flip it with setReverseDirection.

EXERCISE

Tune the speed loop in the simulator

1. In Constants.java, set KS, KV and KA to 0. Run **Simulate Robot Code**.

2. Enable and hold A: the goal is 2 m/s. Note where Drive/Left speed (mps) stops.

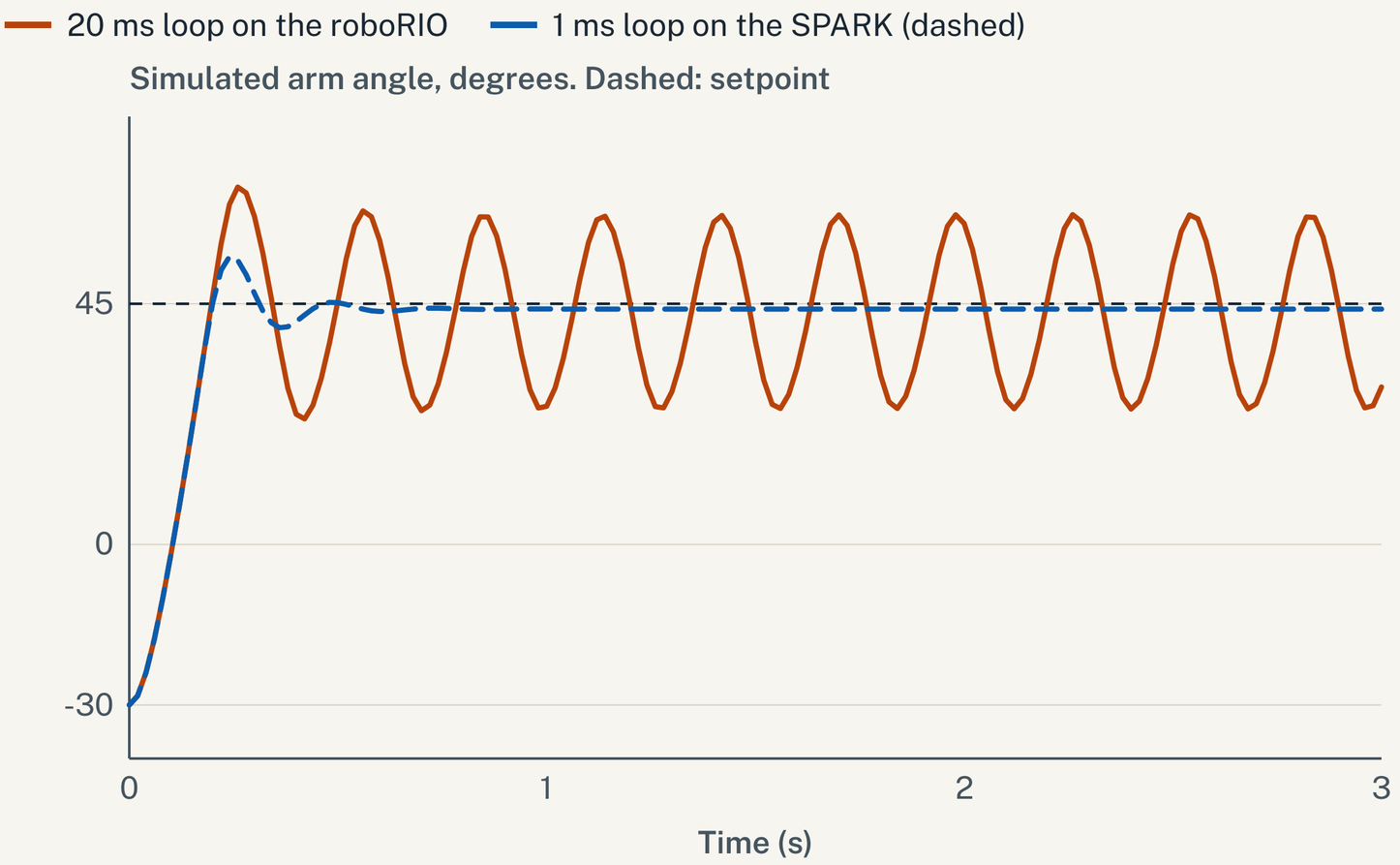
3. Put back KV, then KS, then KA. Hold A after each one.

4. Hold A and Y. Y is a simulated push. Raise KP until the push barely matters.

Done when: with A and Y held, Drive/Left speed (mps) stays within 0.05 of 2 m/s.

PROGRAMMING

An onboard loop decides faster, on fresher data



1. **roboRIO**
   Decides every 20 ms, using a reading that came over CAN.

2. **SPARK**
   REV updates the loop every 1 ms, with the encoder wired right to it.

3. **Same gains**
   At kP 50 the 20 ms loop shakes. The 1 ms loop settles in 0.42 s.

PROGRAMMING

The arm loop can move onto the SPARK MAX

SparkArmSubsystem.java

```
  private static SparkMaxConfig sparkConfig() {
    SparkMaxConfig config = ArmConfig.base();
    config.closedLoop
1       .p(SPARK_KP).d(SPARK_KD)
        .outputRange(-MAX_OUTPUT, MAX_OUTPUT);
    config.closedLoop.feedForward
2       .kCos(KG) // an arm uses kCos, not REV's kG
3       .kCosRatio(1.0 / GEAR_RATIO); // NEO turns to arm turns
    return config;
  }

  private void setGoal(double goal) {
    goalRad = MathUtil.clamp(goal, SOFT_MIN_RAD, SOFT_MAX_RAD);
4   double rotations = armRadToMotorRotations(goalRad);
4   closedLoop.setSetpoint(rotations, ControlType.kPosition);
  }
```

1. **Native units**
   p is duty cycle per NEO rotation. kP 30 is 0.26. Start low.

2. **kCos, not kG**
   WPILib's kG is REV's kCos. REV's kG is for elevators.

3. **kCosRatio**
   NEO rotations to arm rotations: 1 / 60. The sensor must read 0 at flat.

4. **Send it once**
   Goals are in NEO rotations. One message over CAN, then the SPARK does the work.

BEST PRACTICE

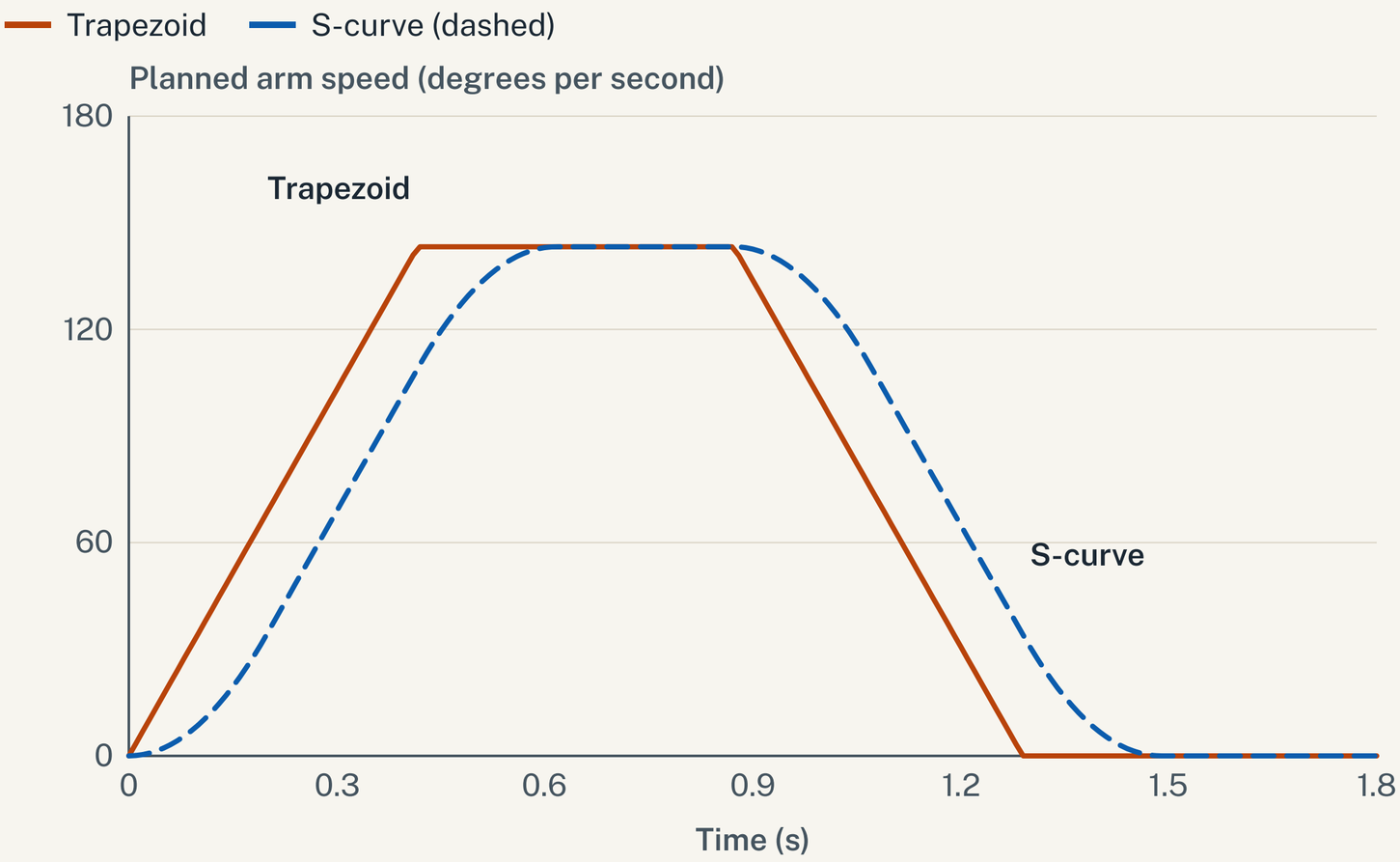
Learn the loop in Java, then move it

> **Get a new loop right on the roboRIO, then move it to the controller.**

Java loops are easy to log, change and try in the simulator. A controller loop is quicker and fresher, but its gains use other units, so you tune it again.

CONTROL THEORY

An S-curve rounds off the corners of a trapezoid



1. **Trapezoid**
   The push switches on and off at once. Those are the sharp corners.

2. **S-curve**
   The push builds over 0.2 s. How fast it builds is called jerk.

3. **The cost**
   This swing takes 1.49 s, not 1.29 s.

4. **Where to find it**
   CTRE's Motion Magic adds it when you set jerk. TrapezoidProfile and MAXMotion stay trapezoids.

WHERE TO GO NEXT

State-space and LQR are the next step up

State-space control describes a mechanism with a small math model, and kV and kA are already one. **LQR** then picks the feedback gains from how much error and how many volts you will accept.

> **The trade-off:** you choose how much error and voltage to accept, not the gains.

> **Already in use:** SysId uses LQR for the P and D gains it suggests.

> **WPILib's advice:** pick the easiest strategy that works on the field.

RECAP

What this deck covered

- The stick chain in order: sign, deadband, curve, scale, slew limit.
- kS, kV and kA, and what SysId's two tests measure.
- Driving by speed with feedforward plus P, after checking encoder direction.
- Where a loop belongs: 20 ms on the roboRIO or 1 ms on the controller.
- How an S-curve differs from a trapezoid, and what state-space adds.

Next: Deck 10 compares the autonomous path planning libraries.

REFERENCE

Official docs



Feedforward in WPILib

docs.wpilib.org



System identification

docs.wpilib.org



SPARK closed loop

docs.revrobotics.com



State-space intro

docs.wpilib.org